

ESTRUCTURAS DE DATOS

# Ayudantía 3: TDAs Lineales

Ignacia Nettle Oliveri  
inettle@usm.cl

BLOQUE 1

# Listas

Listas basadas en Arreglos y Listas Enlazadas

## TDA Lineales

Un TDA Lineal es un modelo de datos donde los elementos se organizan en una **secuencia**, de modo que cada elemento tiene un único predecesor (excepto el primero) y un único sucesor (excepto el último). Los TDAs lineales que veremos en este curso serán:

- ▶ Listas (y sus múltiples implementaciones)
- ▶ Pilas
- ▶ Colas

## Definición

Una lista es una colección secuencial, **finita** y **ordenada** de datos.

- ▶ "Orden" en este contexto refiere a que cada elemento tiene asociado una **posición** dentro de la lista.
- ▶ Para llegar a un elemento  **$i$**  de la lista desde una posición  **$k$** , es necesario pasar por todos los elementos entre ellos.

El comienzo y final de una lista se denominan **cabeza** y **cola** respectivamente.

## Operaciones Básicas

Una lista generalmente es una estructura de datos **dinámica**:

- ▶ Crece al **insertar** nuevos elementos y decrece al **eliminarlos**.
- ▶ Soporta insertar y borrar elementos **en cualquier posición de la lista**.
- ▶ Es posible acceder a cualquier elemento en la lista, tanto para leerlos como modificarlos.
- ▶ Dado un elemento en la lista ("elemento actual"), sobre el cual se llevan a cabo las operaciones, podemos acceder tanto a su predecesor como sucesor.
- ▶ Soporta **reiniciar** la lista (borrar todo su contenido y "empezar de nuevo").

Supondremos **tElem** como el tipo de datos de los elementos de la lista.

```
void clear();
int insert(tElem item);
int append(tElem item);
tElem erase();
void moveToStart();
void moveToEnd();
void next();
void prev();
int length();
int currPos();
void moveToPos(int pos);
tElem getValue();
```

Notar que **prev** y **next** **no producen cambios** si el elemento actual es la cabeza o la cola, respectivamente.

**insert** retorna la posición del elemento insertado, **append** retorna un código de estado y **erase** retorna el valor del elemento eliminado.

# Listas Basadas en Arreglos

## Definición

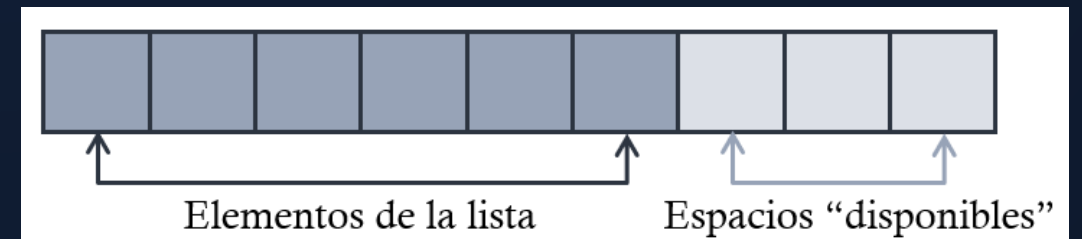
Una forma de implementar el TDA `tLista` es usando un **arreglo base** que almacena los elementos de la lista:

- Cada celda del arreglo almacena un elemento.
- El elemento  $a_i$  es almacenado en la posición  $i$  del arreglo.
- Elementos consecutivos de la lista están en posiciones consecutivas del arreglo.

La cantidad de elementos que tendrá una lista normalmente **no es conocida de antemano**.

Ya que un arreglo es de **tamaño fijo**:

- El máximo tamaño de la lista es igual al tamaño del arreglo.
- Es necesario saber la porción usada del arreglo en todo momento.



# Listas Basadas en Arreglos: **insert**

Al insertar un elemento, es necesario hacer espacio para el elemento entrante.

Si se quiere insertar un elemento en la posición **i**, entonces los elementos ya almacenados desde **i** hasta **n** (siendo **n** el largo actual de la lista) deben ser movidos un espacio a la derecha.

El costo de mover los elementos es, en el peor caso,  $O(n)$

Insertar 23



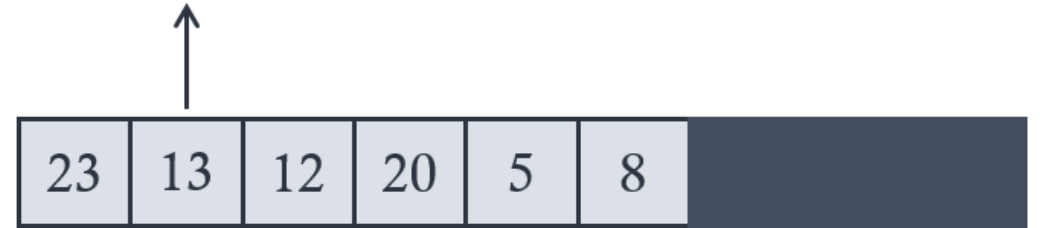
# Listas Basadas en Arreglos: **erase**

Al eliminar un elemento, queda **un espacio vacío**. Ya que las listas son estructuras secuenciales, no pueden haber espacios vacíos entre elementos, por lo que es necesario mover los elementos.

Si se elimina el elemento en la posición  **$i$** , entonces los elementos almacenados desde la posición  **$i+1$**  hasta  **$n$**  deben ser movidos un espacio hacia la izquierda.

El costo de mover los elementos es, en el peor caso,  $O(n)$

Eliminar 13



# Listas Basadas en Arreglos: Tiempos de Ejecución

| Método                   | Tiempo de Ejecución |
|--------------------------|---------------------|
| <code>clear</code>       | $O(1)$              |
| <code>insert</code>      | $O(n)$              |
| <code>append</code>      | $O(1)$              |
| <code>erase</code>       | $O(n)$              |
| <code>moveToStart</code> | $O(1)$              |
| <code>moveToEnd</code>   | $O(1)$              |

| Método                 | Tiempo de Ejecución |
|------------------------|---------------------|
| <code>next</code>      | $O(1)$              |
| <code>prev</code>      | $O(1)$              |
| <code>length</code>    | $O(1)$              |
| <code>currPos</code>   | $O(1)$              |
| <code>moveToPos</code> | $O(1)$              |
| <code>getValue</code>  | $O(1)$              |

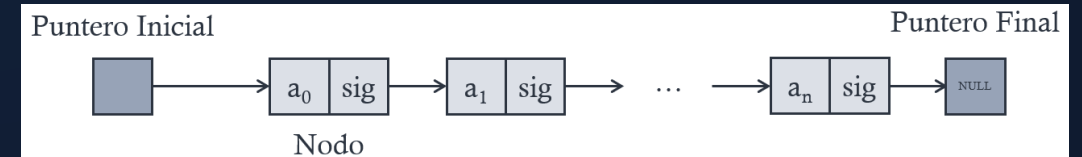


# Listas Enlazadas

## Definición

En este caso, la implementación usa **punteros** para enlazar los nodos de la lista y **memoria dinámica** para insertar y borrar elementos de ella.

- ▶ **Nodos** : almacena un elemento de la lista. **No están necesariamente almacenados de forma contigua** en memoria.
- ▶ **Punteros entre nodos** : permiten enlazar los nodos.



Es **crucial** nunca perder acceso a un nodo, pues se pierde acceso a él y a todos sus nodos sucesores.

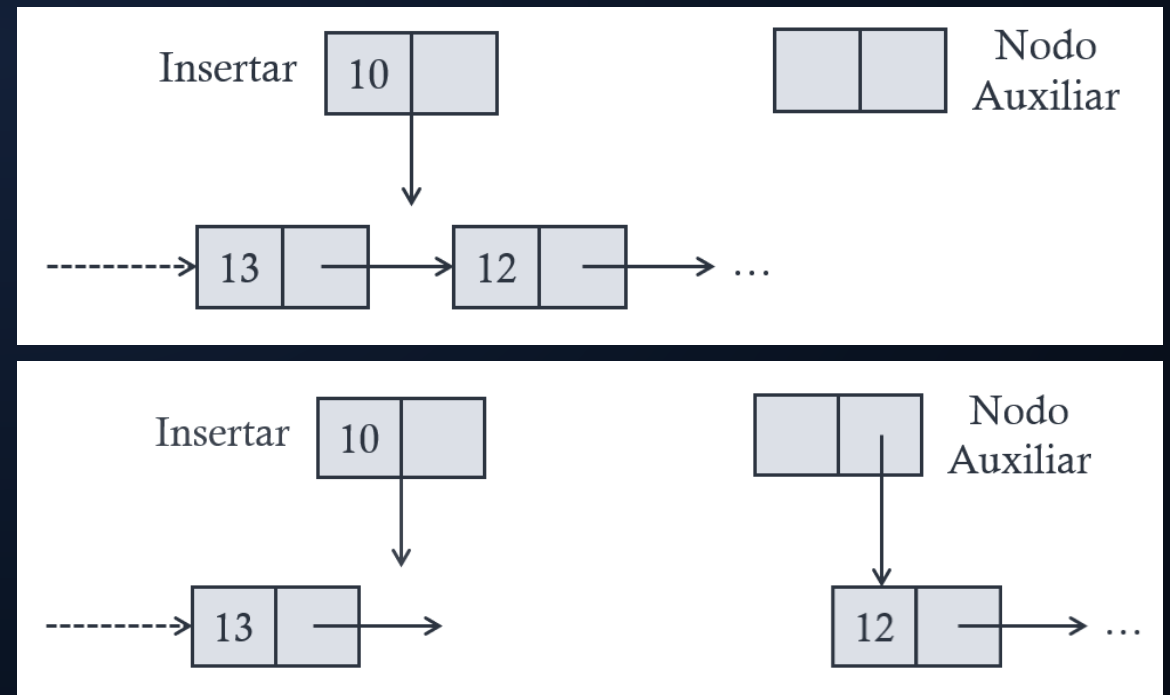
El acceso a un nodo de la lista tiene costo  $O(n)$ , ya que es necesario pasar por todos sus nodos predecesores.

# Listas Enlazadas: **insert**

Para insertar elementos en una lista enlazada, debemos modificar los **punteros** del **predecesor** y **sucesor** del nuevo elemento.

Generalmente, se hace uso de un **nodo auxiliar** que apunta al sucesor para no perder acceso a él.

El costo de insertar un elemento es, en el peor caso,  $O(1)$

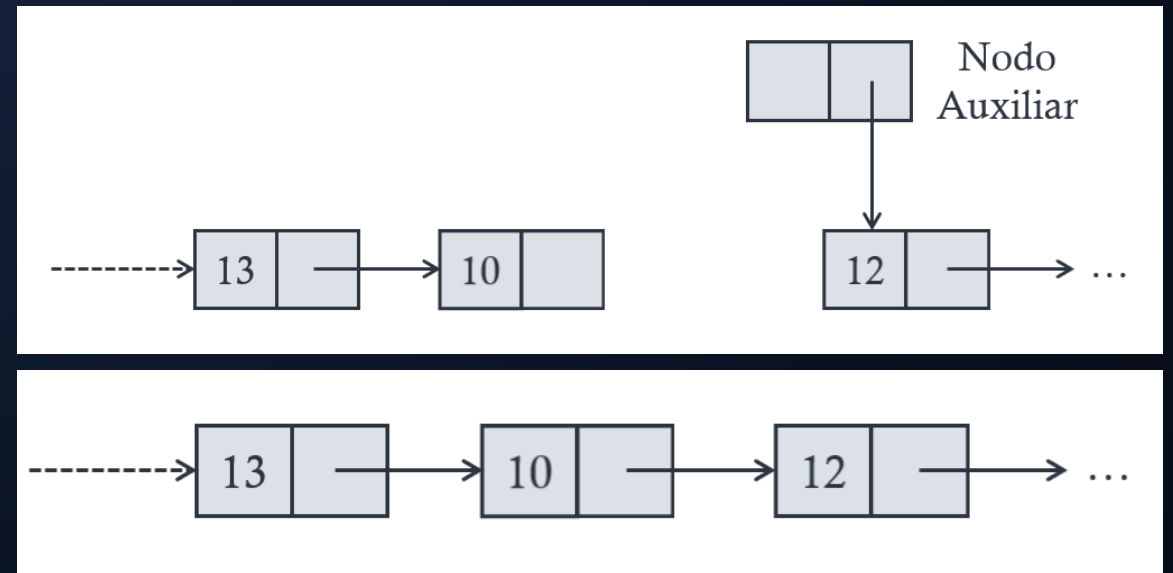


# Listas Enlazadas: **insert**

Para insertar elementos en una lista enlazada, debemos modificar los **punteros** del **predecesor** y **sucesor** del nuevo elemento.

Generalmente, se hace uso de un **nodo auxiliar** que apunta al sucesor para no perder acceso a él.

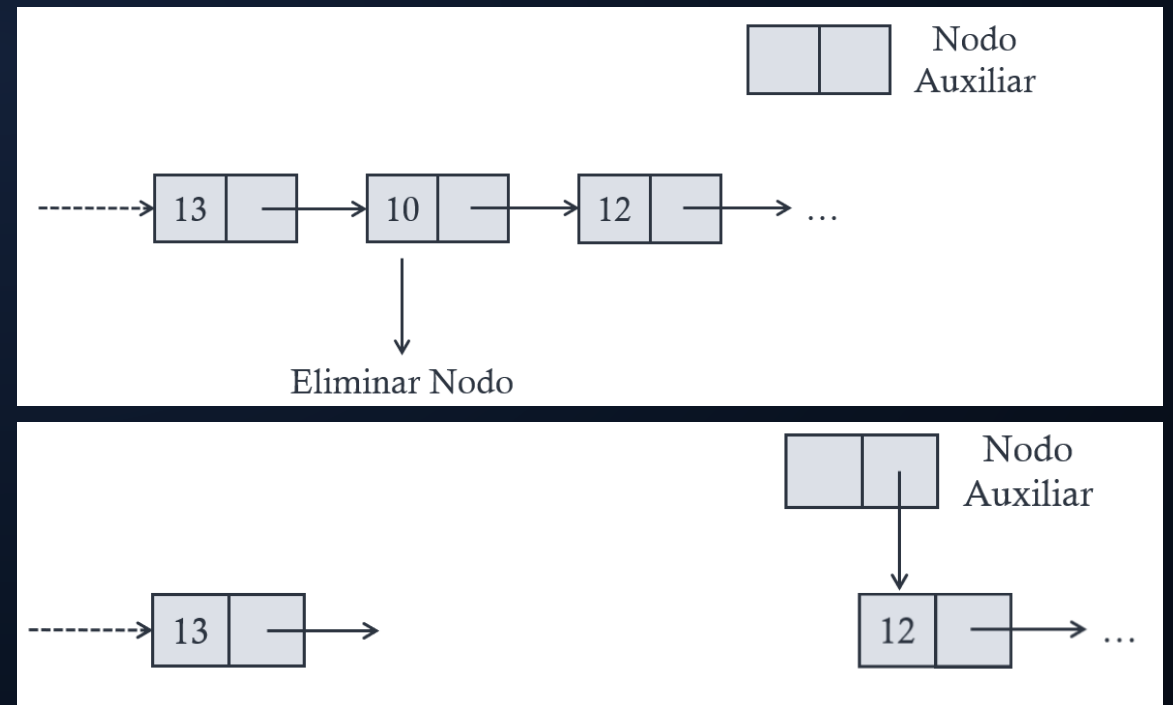
El costo de insertar un elemento es, en el peor caso,  $O(1)$



# Listas Enlazadas: **erase**

Para eliminar un elemento, se sigue el **mismo proceso que el de inserción**: modificar los punteros del predecesor y sucesor del elemento a eliminar, usando un nodo auxiliar para no perder acceso al sucesor.

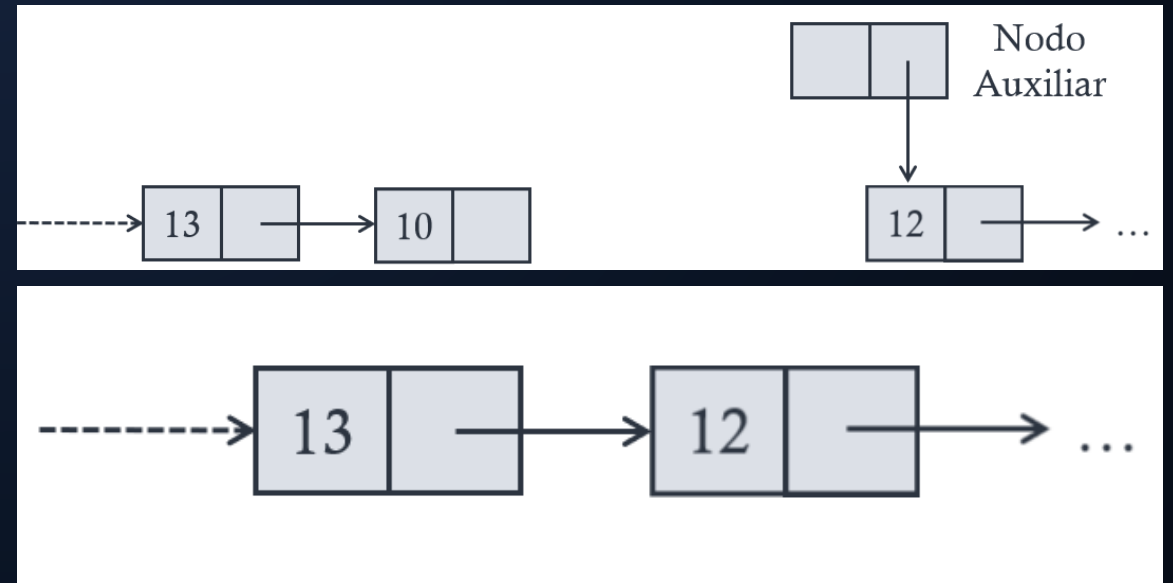
El costo de eliminar un elemento es, en el peor caso  $O(1)$ .



# Listas Enlazadas: **erase**

Para eliminar un elemento, se sigue el **mismo proceso que el de inserción**: modificar los punteros del predecesor y sucesor del elemento a eliminar, usando un nodo auxiliar para no perder acceso al sucesor.

El costo de eliminar un elemento es, en el peor caso  $O(1)$ .



# Listas Enlazadas: Tiempos de Ejecución

| Método                   | Tiempo de Ejecución |
|--------------------------|---------------------|
| <code>clear</code>       | $O(n)$              |
| <code>insert</code>      | $O(1)$              |
| <code>append</code>      | $O(1)$              |
| <code>erase</code>       | $O(1)$              |
| <code>moveToStart</code> | $O(1)$              |
| <code>moveToEnd</code>   | $O(1)$              |

| Método                 | Tiempo de Ejecución |
|------------------------|---------------------|
| <code>next</code>      | $O(1)$              |
| <code>prev</code>      | $O(n)$              |
| <code>length</code>    | $O(1)$              |
| <code>currPos</code>   | $O(1)$              |
| <code>moveToPos</code> | $O(n)$              |
| <code>getValue</code>  | $O(1)$              |

# Lista Basada en Arreglos V/S Lista Enlazada

## Acceso a Elementos

- ▶ Listas Basadas en Arreglos son muy eficientes. En el peor caso, acceder a un elemento de la lista tiene un costo de  $O(1)$ .
- ▶ Listas Enlazadas, en cambio, necesitan iterar sobre cada elemento para llegar al que se quiere consultar. En el peor caso, tiene costo  $O(n)$ .

## Inserción o Eliminación de Elementos

- ▶ Listas Basadas en Arreglos tienen que iterar sobre cada elemento de la lista para reordenarlos, lo que en el peor caso tiene costo  $O(n)$ .
- ▶ Listas Enlazadas, en cambio, modifica punteros en tiempo constante, lo que en el peor caso tiene costo  $O(1)$ .

Escoger una implementación sobre otra dependerá de los requerimientos del programa.

BLOQUE 2

# Ejercicios



# Ejercicio 1

Implementar un programa que permita gestionar una lista de **tareas pendientes**. Cada tarea tiene:

- ▷ Nombre (string)
- ▷ Prioridad (int, menor número implica mayor prioridad)

El programa debe ser capaz de:

- ▷ Mostrar el top 3 de tareas de mayor prioridad.
- ▷ Eliminar la tarea de mayor prioridad.
- ▷ Mostrar todas las tareas pendientes.

**El proceso de inserción de tareas debe tener costo máximo de  $O(1)$ .**

## Ejercicio 2

Implementar un programa que gestione las calificaciones de los estudiantes en un curso de máximo 50 estudiantes.

El programa debe ser capaz de:

- ▶ Obtener todas las notas reprobadas.
- ▶ Obtener el promedio general del curso.
- ▶ Mostrar todas las notas registradas.

**Las consultas para obtener las notas debe tener un costo máximo de  $O(1)$ .**

CIERRE

¡Eso es todo 🧐!